Appointment Scheduler Technical Report

Software Design 1 SDN150S | Project 12

Prepared for Management | 6 November 2026

https://github.com/ZuluBarbie/AppointmentScheduler

Group 12

Noluthando | 260874574

Zilungile Cwathi | 250513447

Phakamani Nicholas Dlamini | 261059742

Sekete Manthata | 260555770

Liyabona Ndevu | 260173436

Okuhle Ntweni | 260016039

Reneloe Wondeful Phakwago | 260633283

Lisakhanya Songqwaba | 261136437

1 Introduction

The Group 12 scheduler lets a user create, edit and delete appointments, set reminders and view a day, week or month. The final implementation also supports recurrence, clash checks and iCalendar export for Project 12 (Cape Peninsula University of Technology, 2026).

2 Sub questions

How can one stored series supply the three views and reveal future clashes? How can reminders alert once per occurrence? How can the displayed appointments be transferred to another calendar?

3 Discussion

Appointment validates the start, end, repeat range and reminder lead time. AppointmentBook stores each series once. It creates Occurrence records for the selected view. Each row refers back to its stored series, so editing or deleting a row affects that series. A repeat end date is inclusive and may be up to 366 days after the start.

Before adding or editing, the book checks whether time intervals overlap on a common occurrence date. Exclusive end times allow adjacent appointments. After validation, the book writes changes to a temporary file and requests atomic replacement where supported (Oracle, n.d.).

SchedulerFrame uses the paired NetBeans JFrame Form for the base layout and export button. Java adds the repeat and reminder controls. A Swing timer checks upcoming occurrences and tracks alerts for the current session. The JDK builds the standalone version and runs tests. Git records the incremental work from the first window to calendar export.

CalendarExport writes one event per displayed occurrence. It converts dates and times from the computer's time zone to UTC, escapes text and folds long UTF-8 lines using the iCalendar specification (Desruisseaux, 2009). The resulting file is a snapshot for import.

4 Design artefacts

4.1 Pseudocode

READ and validate input, repeat end date and reminder lead time

FOR each other appointment with overlapping times

  IF both occur on any candidate date, reject the change

SAVE the new list and refresh the selected view

FOR each upcoming occurrence inside its reminder window

  IF it has not alerted this session, show and mark the alert

ON export, copy the displayed occurrences and choose a file

WRITE one calendar event per row using its occurrence date

4.2 UML class diagram

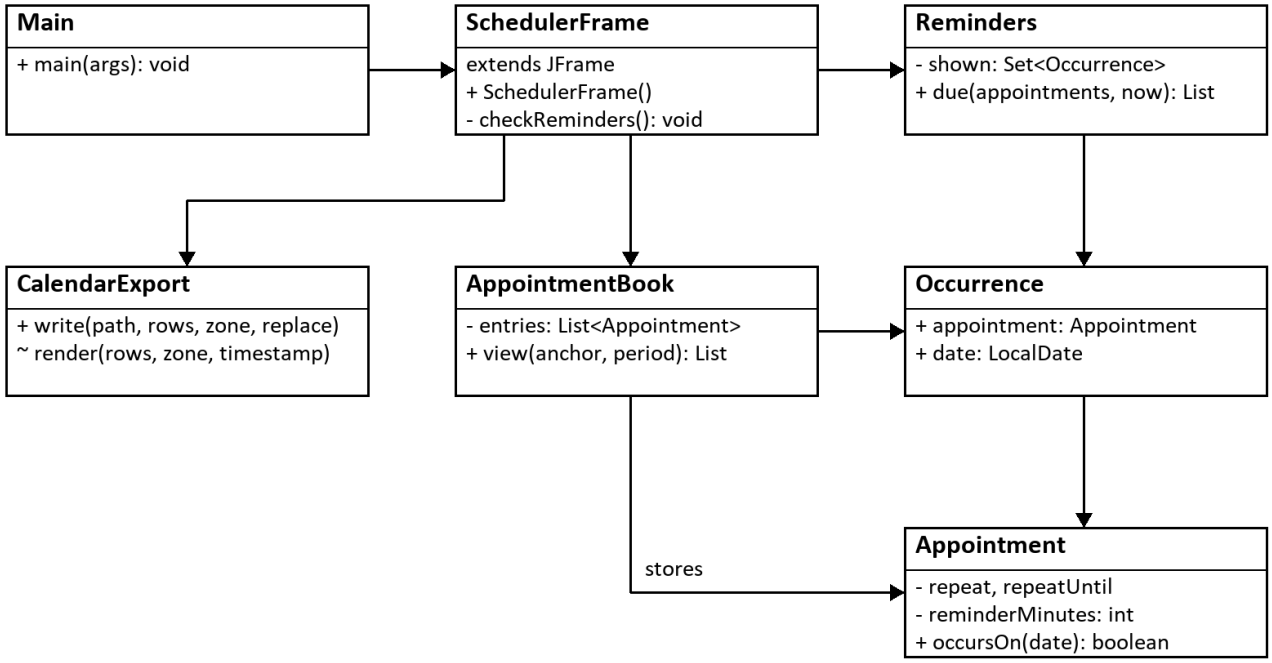


Figure 1. The frame uses dated occurrences for display, reminders and export.

5 Results and analysis

The standalone Day 5 JAR built on Windows. All 64 automated checks passed. Nine export checks covered file framing, date scope, UTC conversion, escaped text, UTF-8 folding, event IDs, safe file replacement and a daylight-saving gap.

NetBeans 31 opened the Day 5 project and displayed its JFrame in Design view. A separate Swing test activated the export button and saved a file through the chooser. A two-day series produced one event in the Day view, and the source file remained intact. The export was verified locally.

Reminders run while the application is open. They can appear again after reopening during the lead time. The exported calendar contains event snapshots. Reimporting can create duplicates. Local files are intended for one user.

6 Conclusion

The scheduler provides the planned appointment workflow and calendar export. The source, report and recording guide are available in the repository. Group 12 must record the demonstration and submit the required materials by 8 November 2026.

References

Cape Peninsula University of Technology (2026) Software Design 1 SDN150S project brief. Course handout.

Desruisseaux, B. (ed.) (2009) Internet Calendaring and Scheduling Core Object Specification (iCalendar). RFC 5545. Available at: https://www.rfc-editor.org/rfc/rfc5545 (Accessed: 6 November 2026).

Oracle (n.d.) Files. Java SE 21. Available at: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/nio/file/Files.html (Accessed: 4 November 2026).

Appendix A Java program excerpts

The complete Java program, paired form and tests are in demos/day5. The entry point is demo.Main. These excerpts show the recurrence rule and how displayed rows become calendar events.

Appointment recurrence

```
public boolean occursOn(LocalDate day) {
    if (day.isBefore(date) || day.isAfter(repeatUntil)) return false;
    return repeat != Repeat.WEEKLY || day.getDayOfWeek() == date.getDayOfWeek();
}
```

CalendarExport event rendering

```
static String render(List<Occurrence> rows, ZoneId zone, Instant timestamp) {
    StringBuilder output = new StringBuilder();
    line(output, "BEGIN:VCALENDAR");
    line(output, "VERSION:2.0");
    line(output, "PRODID:-//SDN150S Group 12//Appointment Scheduler//EN");
    for (Occurrence row : rows) {
        Appointment appointment = row.appointment();
        line(output, "BEGIN:VEVENT");
        line(output, "UID:" + UUID.randomUUID() + "@appointment-scheduler.local");
        line(output, "DTSTAMP:" + UTC.format(timestamp));
        line(output, "DTSTART:" + eventTime(row.start(), zone));
        line(output, "DTEND:" + eventTime(row.date().atTime(appointment.getEnd()), zone));
        line(output, "SUMMARY:" + escape(appointment.getTitle()));
        if (!appointment.getLocation().isEmpty()) {
            line(output, "LOCATION:" + escape(appointment.getLocation()));
        }
        line(output, "END:VEVENT");
    }
    line(output, "END:VCALENDAR");
    return output.toString();
}
```